



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA SOFTWARE

ASIGNATURA:

APLICACIONES WEB

NOMBRE:

IRVIN MARCELO CAJAS IBARRA

FECHA:

LUNES, 15 DE JUNIO DEL 2026

(Semana VII)

1 Introducción

Este proyecto consiste en el desarrollo de una aplicación web utilizando ASP.NET Core MVC, un framework de Microsoft que organiza el código bajo el patrón Modelo-Vista-Controlador. A través de la construcción de un catálogo de libros, se aplicaron conceptos como la validación de datos con Data Annotations, la protección de formularios con AntiForgeryToken y el manejo de rutas mediante controladores y vistas Razor.

2 Entorno y Herramientas

- **IDE:** Visual Studio Code
- **SDK:** .NET 10.0.301
- **Sistema operativo:** Windows
- **Control de versiones:** Git + GitHub

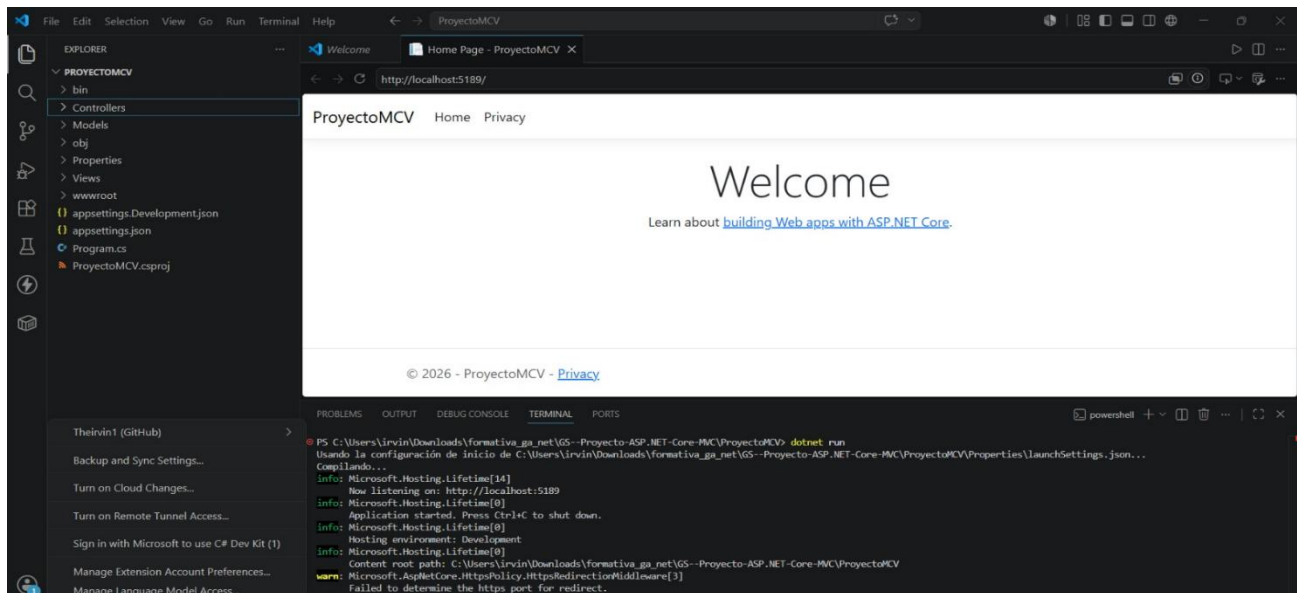
3 Paso 1 — Creación del Proyecto

3.1 Comando ejecutado

- `dotnet new mvc -n PROYECTOMCV`
- `cd MiPrimerMVC`
- `dotnet run`

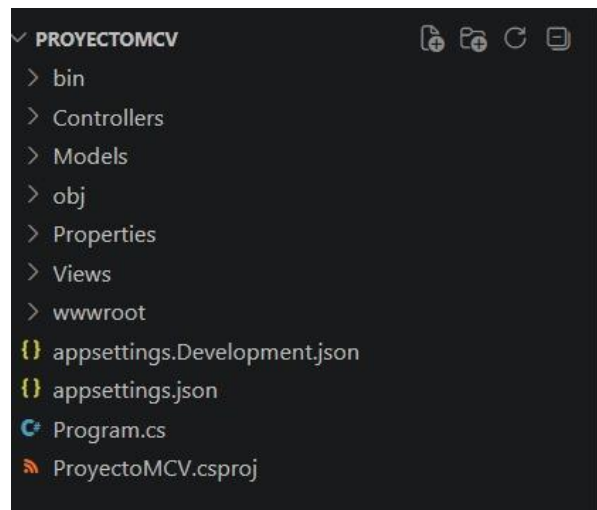
3.2 Evidencia en terminal

Se ejecutó el comando `dotnet new mvc -n PROYECTOMCV` en la terminal, lo cual generó automáticamente la estructura base del proyecto. A continuación, se ejecutó `dotnet run` y el servidor quedó escuchando en localhost, confirmando que el proyecto fue creado sin errores.



3.3 Estructura de carpetas generada

Estructura (Jerarquía) de la carpeta.



4 Paso 2 — Modelo Producto con Data Annotations y ProductoController

4.1 Código del Modelo

```
using System.ComponentModel.DataAnnotations;

namespace ProyectoMCV.Models
{
    public class Producto
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El título es obligatorio")]
        [StringLength(150, ErrorMessage = "Máximo 150 caracteres")]
        [Display(Name = "Título")]
        public string? Titulo { get; set; }

        [Required(ErrorMessage = "El autor es obligatorio")]
        [StringLength(100, ErrorMessage = "Máximo 100 caracteres")]
        [Display(Name = "Autor")]
        public string? Autor { get; set; }

        [Required(ErrorMessage = "El precio es obligatorio")]
        [Range(0.01, 500, ErrorMessage = "El precio debe ser entre 0.01 y
500")]
        [Display(Name = "Precio")]
        public decimal Precio { get; set; }

        [Required(ErrorMessage = "El número de páginas es obligatorio")]
        [Range(1, 5000, ErrorMessage = "Debe tener entre 1 y 5000
páginas")]
        [Display(Name = "Páginas")]
        public int Paginas { get; set; }

        [Required(ErrorMessage = "El ISBN es obligatorio")]
        [StringLength(13, MinimumLength = 10, ErrorMessage = "El ISBN debe
tener entre 10 y 13 caracteres")]
        [Display(Name = "ISBN")]
        public string? ISBN { get; set; }
    }
}
```

4.2 Archivo Producto.cs en el editor

Se creó el archivo Producto.cs dentro de la carpeta Models. En él se definió la clase Producto con los campos Titulo, Autor, Precio, Paginas e ISBN, cada uno decorado con sus respectivas Data Annotations para validación.

4.3 Código del Controlador

```
using Microsoft.AspNetCore.Mvc;
using ProyectoMCV.Models;

namespace ProyectoMCV.Controllers
{
    public class ProductoController : Controller
    {
        private static List<Producto> _productos = new List<Producto>();

        public IActionResult Index()
        {
            return View(_productos);
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Producto producto)
        {
            if (ModelState.IsValid)
            {
                producto.Id = _productos.Count + 1;
                _productos.Add(producto);
                return RedirectToAction("Index");
            }
            return View(producto);
        }
    }
}
```

4.4 Archivo ProductoController.cs en el editor

Se creó el archivo ProductoController.cs dentro de la carpeta Controllers. El controlador contiene tres acciones: Index que retorna la lista de libros, Create GET que muestra el formulario vacío, y Create POST que recibe los datos del formulario, valida el modelo y en caso de ser válido lo agrega a la lista y redirige al Index.

5 Paso 3 — Vistas Razor

5.1 Código de la Vista Index

```
@model IEnumerable<ProyectoMCV.Models.Producto>

<h2>Catálogo de Libros</h2>

<a asp-action="Create" class="btn btn-primary mb-3">+ Agregar Libro</a>

<table class="table table-bordered">
    <thead>
        <tr>
            <th>Título</th>
            <th>Autor</th>
            <th>Precio</th>
            <th>Páginas</th>
            <th>ISBN</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var libro in Model)
        {
            <tr>
                <td>@libro.Titulo</td>
                <td>@libro.Autor</td>
                <td>$@libro.Precio</td>
                <td>@libro.Paginas</td>
                <td>@libro.ISBN</td>
            </tr>
        }
    </tbody>
</table>
```

5.2 Vista Index en el navegador

Se creó la vista Index.cshtml dentro de Views/Producto/. En el navegador se puede ver la tabla con las columnas Título, Autor, Precio, Páginas e ISBN, junto con el botón para agregar un nuevo libro. Inicialmente la tabla aparece vacía hasta que se registre el primer libro.

5.3 Código de la Vista Create

```
@model ProyectoMCV.Models.Producto

<h2>Agregar Libro</h2>

<form asp-action="Create" method="post">
    @Html.AntiForgeryToken()

    <div class="mb-3">
        <label asp-for="Titulo" class="form-label"></label>
        <input asp-for="Titulo" class="form-control" />
        <span asp-validation-for="Titulo" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Autor" class="form-label"></label>
        <input asp-for="Autor" class="form-control" />
        <span asp-validation-for="Autor" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Precio" class="form-label"></label>
        <input asp-for="Precio" class="form-control" />
        <span asp-validation-for="Precio" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Paginas" class="form-label"></label>
        <input asp-for="Paginas" class="form-control" />
        <span asp-validation-for="Paginas" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="ISBN" class="form-label"></label>
        <input asp-for="ISBN" class="form-control" />
        <span asp-validation-for="ISBN" class="text-danger"></span>
    </div>
</form>
```

```
</div>

<button type="submit" class="btn btn-success">Guardar</button>
<a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>
```

5.4 Formulario Create en el navegador

Se creó la vista Create.cshtml con un formulario que incluye un campo para cada propiedad del modelo. El formulario utiliza `@Html.AntiForgeryToken()` para protegerse contra ataques CSRF, y cada campo cuenta con su etiqueta de error para mostrar los mensajes de validación.

5.5 Catálogo de Libros

Catálogo de Libros

+ Agregar Libro

Título	Autor	Precio	Páginas	ISBN
--------	-------	--------	---------	------

5.6 Agregar Libro

Agregar Libro

Título

Autor

Precio

Páginas

ISBN

Guardar

Cancelar

6 Paso 4 — Validación Server-Side

6.1 Errores de validación en el formulario

Al intentar enviar el formulario con campos vacíos o con datos fuera de rango, el servidor rechaza la solicitud porque `ModelState.IsValid` retorna `false`. La vista vuelve a renderizarse mostrando los mensajes de error en rojo debajo de cada campo que no pasó la validación.

Agregar Libro

Título

El título es obligatorio

Autor

El autor es obligatorio

Precio

The value "" is invalid.

Páginas

The value "" is invalid.

ISBN

El ISBN es obligatorio

Guardar Cancelar

6.2 Libro creado exitosamente

Tras completar el formulario con datos válidos y presionar Guardar, el controlador agrega el libro a la lista y redirige a la vista Index. En ella se puede ver el nuevo registro apareciendo en la tabla con todos sus datos correctamente guardados.

Catálogo de Libros

[+ Agregar Libro](#)

Título	Autor	Precio	Páginas	ISBN
Cien años de soledad	Gabriel García Márquez	\$15,99	496	978-030747472

7 Repositorio GitHub

Link: <https://github.com/Theirvin1/GS--Proyecto-ASP.NET-Core-MVC>

8 Conclusión

El desarrollo de este proyecto permitió comprender de manera práctica cómo funciona el patrón MVC y cómo cada componente cumple un rol específico dentro de la aplicación. La separación entre modelo, vista y controlador facilita la organización del código y su mantenimiento. Además, el uso de Data Annotations demostró ser una forma sencilla y efectiva de implementar validaciones sin necesidad de código adicional.